7-1 什么是依赖注入

需求场景

你的一个网站有两个查询接口，一个是图书列表接口，一个是用户列表接口。两个接口有相同的分页查询逻辑。此时你该如何实现？

通常，你的实现方案可能是下面这个样子，

```
from fastapi import FastAPI

app = FastAPI()


BOOKS = [{"id": i, "name": f"book{i}", "status": i % 4 != 0} for i in range(1,
11)]
USERS = [{"id": i, "name": f"user{i}", "status": i % 4 != 0} for i in range(1,
11)]


@app.get("/api/books")
def get_books(page: int = 1, size: int = 2, status: bool = True):   # 需要定义三个
参数 page\size\status
    books = [b for b in BOOKS if b["status"] == status]
    return books[(page - 1) * size:page * size]


@app.get("/api/users")
def get_books(page: int = 1, size: int = 2, status: bool = True):   # 需要重复再次
定义三个参数
    users = [u for u in USERS if u["status"] == status]
    return users[(page - 1) * size:page * size]
```

FastAPI使用依赖注入解决

- 对于上面的代码重复问题，我们可以使用依赖注入解决
- 依赖注入其实就是英文单词 Dependency Injection 的翻译，它是一种非常简单且直观的工具。简单来说，就是当你的代码中的一个参数的值需要依赖其他条件时，可以使用依赖注入。当你使用了它之后，FastAPI就会在需要使用这个参数的时候，自动帮你执行它的依赖条件来获取结果。

示例1：依赖注入的使用

- 首先，从FastAPI引入`Depends`
- 然后，定义依赖条件，即定义一个函数（可调用对象即可），函数的形参数我们需要的参数，返回值是三个参数组成的字典
- 最后，在路径函数内使用 `Depends(common_params)`，这样一来FastAPI就知道，`commons`这个字典依赖`common_params`函数的返回值，即`commons`字典就是`common_params`函数的返回值。

- 像`common_params`函数被称为依赖条件，只要是可调用对象就可以当依赖条件。
- 依赖条件中可以使用Path\Query等等获取参数的方式。

```
from fastapi import FastAPI, Depends

app = FastAPI()


BOOKS = [{"id": i, "name": f"book{i}", "status": i % 4 != 0} for i in range(1,
11)]
USERS = [{"id": i, "name": f"user{i}", "status": i % 4 != 0} for i in range(1,
11)]


# 定义依赖条件
def common_params(page: int = 1, size: int = 2, status: bool = True):
    return {
        "page": page,
        "size": size,
        "status": status,
    }


@app.get("/api/books")
def get_books(commons: dict = Depends(common_params)):  # 使用依赖条件
    page = commons["page"]
    size = commons["size"]
    status = commons["status"]
    books = [b for b in BOOKS if b["status"] == status]
    return books[(page - 1) * size:page * size]


@app.get("/api/users")
def get_users(commons: dict = Depends(common_params)):
    page = commons["page"]
    size = commons["size"]
    status = commons["status"]
    users = [u for u in USERS if u["status"] == status]
    return users[(page - 1) * size:page * size]
```

依赖注入的用途：

- 共享一块相同逻辑的代码块
- 共享数据库连接
- 权限认证，登录状态认证
- 等等等

7-2 依赖注入嵌套使用

依赖注入是非常强大的，比如说，它可以支持嵌套使用，且嵌套深度不受限制。

示例：两层嵌套依赖注入

- 路径函数get_name需要的形参`username_or_nickname`有依赖条件，所以FastAPI会调用`username_or_nickname_extractor`
- 执行`username_or_nickname_extractor`的时候，发现它也有依赖条件，所以FastAPI会调用`username_extractor`
- 按照这个顺序，依次获取每个有依赖条件的参数的结果。最终，在路径函数内获取最终的结果。

```
from typing import Union

from fastapi import Depends, FastAPI

app = FastAPI()


def username_extractor(username: Union[str, None] = None):
    return username


def username_or_nickname_extractor(
    username: str = Depends(username_extractor),
    nickname: Union[str, None] = None,
):
    if not username:
        return nickname
    return username


@app.get("/name")
def get_name(username_or_nickname: str =
Depends(username_or_nickname_extractor)):
    return {"username_or_nickname": username_or_nickname}
```

7-3 依赖注入的缓存现象

很多时候，我们定义的依赖条件会被执行多次，这种场景下，FastAPI默认只会执行一次依赖条件。但我们也可以执行不使用缓存。

示例1：依赖注入的缓存现象

- 依赖条件`get_num`被依赖了两次，但是你会发现其内部打印语句只打印了一次。也就是说，第二次使用这个依赖条件时FastAPI并没有真正执行这个函数，而是直接使用了第一次执行的结果，这就是依赖注入的缓存现象。

```python
from fastapi import Depends, FastAPI

app = FastAPI()


def get_num(num: int):
    print("get_num被执行了")
    return num


@app.get("/")
async def get_results(num1: int = Depends(get_num), num2: int =
Depends(get_num)):
    return {"num1": num1, "num2": num2}
```

实例2：依赖注入不使用缓存

- 默认 `use_cache` 字段是True，如果在第二次使用依赖注入不想使用缓存，将此字段的值设为False即可。
- 需要注意，

```python
@app.get("/")
async def get_results(num1: int = Depends(get_num), num2: int = Depends(get_num,
use_cache=False)):
    return {"num1": num1, "num2": num2}
```

示例3：缓存现象存在缓存嵌套中

- 依赖注入嵌套使用时，子依赖如果被使用多次也会存在缓存现象，解决办法就是第二次使用子依赖时使用use_cache=False

```python
from fastapi import Depends, FastAPI

app = FastAPI()

# 这个子依赖被使用两次
def get_num(num: int):
    print("get_num被执行了")
    return num


def get_result1(num: int = Depends(get_num)):
    return num * num


# 第二次使用get_num时，设置use_cache=False则不使用缓存
def get_result2(num: int = Depends(get_num, use_cache=False)):
    return num * num * num


@app.get("/")
async def get_results(result1: int = Depends(get_result1), result2: int =
Depends(get_result2)):
```

```
    return {"result1": result1, "result2": result2}
```

总结：

- 在一个请求中，如果依赖注入条件被使用了多次，则只有第一次会真正执行内部代码，然后将其返回值缓存起来，后面再次使用它则直接获取缓存结果，不会再次执行其内部代码。
- 如果不想使用依赖注入缓存，则可以在这个依赖条件第二次被使用时，设置`use_cache=False`即可。

7-4 路径装饰器和全局依赖注入

需求场景

- 有的时候，我们想使用依赖注入，但并不希望它有返回值，那此时就不能在路径函数内使用`Depends()`了，那该如何呢？

FastAPI的解决方式

不需要返回值的依赖注入，可以在路径装饰器中使用。

示例1：使用依赖注入校验访问权限

- 在路径装饰器中可以使用依赖注入，使用字段`dependencies`，它的值需要是一个包含`Depands()`的序列，比如列表或元组。
- `dependencies`中每个依赖条件不需要返回值，就是有返回值也不会使用。

```
from fastapi import Depends, FastAPI, Header, HTTPException


app = FastAPI()


def verify_token(x_token: str = Header()):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400, detail="X-Token header invalid")
    return x_token


@app.get("/items/", dependencies=[Depends(verify_token)])
def get_items():
    return [{"item": "Foo"}, {"item": "Bar"}]
```

示例2：全局使用依赖注入校验访问权限

- 如果我们希望系统中的所有路由接口都默认有依赖注入，此时我们没有必要每个接口都重复设置一遍依赖注入
- 只需要在实例化FastAPI时，通过`dependencies`，需要注意的是，这个参数接收的值依然是一个包含多个可依赖对象的列表。

```
from fastapi import Depends, FastAPI, Header, HTTPException


def verify_token(x_token: str = Header()):
    if x_token != "fake-super-secret-token":
        raise HTTPException(status_code=400, detail="X-Token header invalid")


app = FastAPI(dependencies=[Depends(verify_token)])


@app.get("/items/")
def get_items():
    return [{"item": "Portal Gun"}, {"item": "Plumbus"}]


@app.get("/users/")
def get_users():
    return [{"username": "Rick"}, {"username": "Morty"}]
```

7-5 基于类的依赖注入

在7-1节中，我们通过定义依赖条件，解决了通用查询参数重复定义的问题。其中，我们定义的依赖条件是普通的函数。

但其实，只要是可调用的对象，都可以当做依赖条件，比如类。类实例化其实就是类的调用。

示例1：类形式的依赖条件

- `CommonQueryParams()`就会实例化出来一个对象，三个通用参数就会保存在在对象的三个属性上。
- 类中定义的`__init__`方法，用来保存变量属性。

```
from fastapi import Depends, FastAPI

app = FastAPI()


BOOKS = [{"id": i, "name": f"book{i}", "status": i % 4 != 0} for i in range(1,
11)]


# 定义依赖条件
class CommonQueryParams:
    def __init__(self, page: int, size: int, status: bool):
```

```
        self.page = page
        self.size = size
        self.status = status


@app.get("/api/books")
def get_books(commons: CommonQueryParams = Depends(CommonQueryParams)):     # 使用依赖条件

    page = commons.page
    size = commons.size
    books = [b for b in BOOKS if b["status"] == commons.status]
    return books[(page - 1) * size:page * size]
```

示例2：类形式依赖注入的简化用法

- 上面我们使用类形式的依赖注入，看起来优点冗余对吧：`commons: CommonQueryParams = Depends(CommonQueryParams)`
- 对于这种情况，有一个简写方式：`commons: CommonQueryParams = Depends()`，此时fastapi知道Depends括号内依赖的是什么。

```
# 定义依赖条件
class CommonQueryParams:
    def __init__(self, page: int, size: int, status: bool):
        self.page = page
        self.size = size
        self.status = status


@app.get("/api/books")
def get_books(commons: CommonQueryParams = Depends()):
    pass
```

7-6 基于对象的依赖注入

需求场景

上面我们可以定义基于函数，或者基于类形式的依赖条件，但是他们有一个共同的缺点：写死不可变的，不支持参数化。

比如，我们有一个需求，要求我们检查查询参数中的字段是否包含指定的文本，并且被检查的文本可以通过参数的形式调整。

示例1：检查指定的文本是否在查询参数q中

- 缺点：代码重复，不够灵活优雅

```
from fastapi import  FastAPI

app = FastAPI()
```

```
@app.get("/hello")
def hello_check(q: str):
    exists = "hello" in q
    return {"exists": exists}


@app.get("/world")
def world_check(q: str):
    exists = "world" in q
    return {"exists": exists}
```

FastAPI的解决方式

对于上面的的需求，我们可以使用基于对象的依赖注入，只要在类下面实现`__call__`方法，该对象就可以被调用，基于可以当做依赖条件

示例2：基于对象的依赖条件，参数化

- 对象`FixedContentQueryChecker("hello")`当依赖条件，它被调用时就会执行其魔法方法`__call__`，就会判断查询参数q中是否包含待检查的文本信息。

```
from fastapi import Depends, FastAPI

app = FastAPI()


class FixedContentQueryChecker:
    def __init__(self, fixed_content: str):     # 实例化对象是执行
        self.fixed_content = fixed_content

    def __call__(self, q: str = "") -> bool:    # 对象被调用时执行
        return self.fixed_content in q


@app.get("/hello")
def hello_check(exists: bool = Depends(FixedContentQueryChecker("hello"))):
    return {"exists": exists}


@app.get("/world")
def world_check(exists: bool = Depends(FixedContentQueryChecker("world"))):
    return {"exists": exists}
```

7-7 依赖注入使用yield

- fastapi的依赖注入非常强大，它还可以具备上下文管理器的功能。

- 想要在依赖注入中实现上下文管理器，我们可以使用 `yield`
- 比如，我们想要在进入路径操作函数时通过依赖获取一个操作数据库的连接，请求结束后关闭这个db连接

示例1：依赖注入中使用yield

- 这样写的好处是，在路径操作函数结束时，会自动关闭db连接回收资源。及时在路径函数会出现异常报错，最终也会关闭连接。

```
from fastapi import Depends, FastAPI


app = FastAPI()


def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()


@app.get("/books/")
def books(db: Depend(get_db)):
    # db.execute("select * form books")   db操作
    pass
```

7-8 补充上下文管理器

上下文管理协议需要实现两个功能：

- `__enter__`，进入上下文时运行，并返回当前对象。如果with语句有as关键词存在，返回值会绑定在as后的变量上。
- `__exit__`，退出上下文时运行

说到上下文管理器就不得不提 `with`，它的一个常见使用场景如下：

- open函数会返回一个文件类型变量，这个文件类实现了上下文管理协议，而with语句就是为支持上下文管理器而存在的。

```
with open("a.txt", "r") as f:
    content = f.read()
```

- 进入with语句块时，就会执行文件类的`__enter__`返回一个文件对象，并赋值给变量 `f`
- 从with语句块出来时，机会执行文件类的`__exit__`，在其内部实现 f.close()，所以使用者就不需要在手动关闭文件对象了。

示例1：手动实现上下文管理器

```
class MyFile():
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        print("进入with...")
        return self     # 返回的数据赋值给p

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("退出with")


with MyWith('jack') as p:
    print(p.name)
```

示例2：contextlib 模块实现上下文管理器

- 使用装饰器 contextmanager
- get_file函数内部，yield语句前的代码在进入with语句时执行，yield的值赋值给 as后面的变量，
- yield后面的代码在退出with语句时执行

```
import contextlib


@contextlib.contextmanager
def get_file(filename: str):
    file = open(filename, "r", encoding="utf-8")
    yield file
    file.close()


with get_file('oauth2.py') as f:
    print(f.read())
```